



Aurora Report

The complete guide to using it in your project

Version 1.4.0

Updated 22 September 2026

Source github.com/chinnu-82/chinnu_report

Contents

1. Install

2. Connect it to Playwright

3. Update your tests

4. Recording steps

5. Screenshots

6. Videos and traces

7. Soft checks, notes and data

8. Test metadata: severity, owner, issues, tags

9. Error capturing

10. Real-world patterns

11. Configuration

12. Configuration recipes

13. Running in CI

14. Reading the report

15. Using results.json

16. Troubleshooting

17. Cheat sheet

Aurora Report: Using It in Your Project

This guide shows how to add Aurora Report to an existing Playwright project and get the most out of it. Every section has examples you can copy.

 **Prefer to print or read offline?** [Download the PDF](#) — the same guide, 45 pages.
New to Aurora? The [Tutorial](#) walks you through the demo report first, with screenshots and flowcharts.
This guide is the reference you come back to.

1. Install

Aurora is not on the public npm registry, so `npm install aurora-report` on its own will not find it. Pick one of these three.

a) Straight from GitHub — the simplest

```
npm install --save-dev @playwright/test github:chinnu-82/chinnu_report
```

It lands in `node_modules` as `aurora-report`, with no build step. Pin it so upgrades are deliberate:

```
npm install --save-dev github:chinnu-82/chinnu_report#<commit-sha>
```

b) A bundled tarball — no network needed, exact bytes pinned

In a checkout of the Aurora repo:

```
git clone https://github.com/chinnu-82/chinnu_report.git
cd chinnu_report
npm pack          # creates aurora-report-1.1.0.tgz
```

Copy that file into your project (for example `vendor/`), commit it, and reference it with a **relative** path so it works on every machine:

```
{
  "devDependencies": {
    "@playwright/test": "^1.63.0",
    "aurora-report": "file:vendor/aurora-report-1.1.0.tgz"
  }
}
```

Then run `npm install`. Anyone cloning your repo gets the reporter with no extra steps. The [Shopify store example](#) is set up this way.

c) Your own npm registry

With Artifactory, GitHub Packages or Verdaccio, `npm publish` it once and install it by name like any other dependency.

Avoid `npm link` and folder links such as `"file:../TestReport"`. They point at Aurora's own `node_modules`, which can load a second copy of Playwright and fail with "Requiring @playwright/test second time". The options above don't have this problem.

Requirements: Node 18 or newer, and `@playwright/test` 1.40 or newer. Aurora was tested with 1.63.

To upgrade later: re-run the GitHub install, or `npm pack` a fresh tarball and reinstall it.

2. Connect it to Playwright

Option A: `withAurora()` (recommended)

Wrap your existing config. Your other reporters and `use` settings stay as they are.

JavaScript (CommonJS)

```
// playwright.config.js
const { defineConfig, devices } = require('@playwright/test');
const { withAurora } = require('aurora-report');

module.exports = withAurora(
  defineConfig({
    testDir: './tests',
    retries: 1,
    projects: [{ name: 'chromium', use: { ...devices['Desktop Chrome'] } }],
  }),
  { title: 'Checkout - E2E' },
);
```

TypeScript

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';
import { withAurora } from 'aurora-report';

export default withAurora(
  defineConfig({
    testDir: './tests',
    retries: process.env.CI ? 2 : 0,
    reporter: [['list'], ['junit', { outputFile: 'results.xml' }]], // kept
    projects: [
      { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
      { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    ],
  }),
  {
    title: 'Checkout - E2E',
    environment: { Build: process.env.BUILD_NUMBER ?? 'local' },
  },
);
```

ES modules (`"type": "module"` in package.json)

```
// playwright.config.js
import { defineConfig } from '@playwright/test';
import { withAurora } from 'aurora-report';

export default withAurora(defineConfig({ testDir: './tests' }), { title: 'My App' });
```

`withAurora()` does three things:

1. It adds the Aurora reporter after your existing reporters. If you have none, it adds `list` so you still see console output.
2. It sets `use.video` and `use.trace` from Aurora's `capture` settings. **Values you set yourself in `use` always win.**
3. It passes the options to your test workers so step screenshots follow your settings.

Option B: add the reporter yourself

Use this if you don't want Aurora to change your `use` settings.

```
// playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  use: {
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'retain-on-failure',
  },
  reporter: [
    ['list'],
    ['aurora-report/src/reporter.js', { title: 'My App', outputDir: 'reports/aurora', open:
'never' }],
  ],
});
```

Run it

```
npx playwright test
```

The last lines of output show where the report is:

```
🌟 Aurora report 42 passed · 2 failed · 1 flaky · 3 skipped
file:///C:/work/my-app/aurora-report/index.html
```

To open the latest report at any time:

```
npx aurora-report open          # uses outputDir from your config  
npx aurora-report open reports/aurora # or a specific folder
```

3. Update your tests

Existing tests work without changes

Tests that import from `@playwright/test` show up in the report. You get:

- a story built from your `test.step()` calls, the actions inside them and your assertions
- the plain-English "Why it failed" explanation
- screenshots, videos and traces that Playwright attaches (according to your `use` settings)

Switch the import to get everything

Change **one line** per spec file:

```
- import { test, expect } from '@playwright/test';  
+ import { test, expect } from 'aurora-report';
```

```
- const { test, expect } = require('@playwright/test');  
+ const { test, expect } = require('aurora-report');
```

You now also get:

- the `report` fixture: steps with screenshots, soft checks, notes, data and metadata
- automatic capture of console errors, page errors and failed requests
- a failure screenshot and the page HTML for every failed test

Aurora's `test` is Playwright's `test` with extra fixtures. `test.describe`, `test.skip`, `test.use`, hooks and everything else work the same way.

Tip: to switch a whole project at once, search and replace `from '@playwright/test'` with `from 'aurora-report'` in your spec files. Keep type-only imports such as `import type { Page } from '@playwright/test'`.

4. Recording steps

A step is one line in the test story. By default Aurora takes a screenshot when each step ends.

Basic step

```
import { test, expect } from 'aurora-report';

test('user can log in', async ({ page, report }) => {
  await report.step('Open the login page', async () => {
    await page.goto('/login');
  });

  await report.step('Enter credentials', async () => {
    await page.getByLabel('Email').fill('ada@example.com');
    await page.getByLabel('Password').fill('secret');
  });

  await report.step('Submit the form', async () => {
    await page.getByRole('button', { name: 'Sign in' }).click();
  });

  await report.step('Dashboard is visible', async () => {
    await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
  });
});
```

One-line steps

```
await report.step('Open the home page', () => page.goto('/'));
await report.step('Accept cookies', () => page.getByRole('button', { name: 'Accept' }).click());
```

Steps that return a value

`report.step` returns whatever your function returns:

```
const orderId = await report.step('Place the order', async () => {
  await page.getByRole('button', { name: 'Pay now' }).click();
  return page.getByTestId('order-id').innerText();
});

await report.step(`Order ${orderId} appears in history`, async () => {
  await page.goto('/orders');
  await expect(page.getByText(orderId)).toBeVisible();
});
```

Nested steps

Child steps are indented in the story. **A parent step doesn't take its own screenshot if one of its children already did**, so you don't get the same picture twice.

```
await report.step('Checkout', async () => {
  await report.step('Fill shipping address', async () => {
    await page.getByLabel('Street').fill('1 Main St');
    await page.getByLabel('City').fill('Springfield');
  });
  await report.step('Choose express delivery', async () => {
    await page.getByLabel('Express').check();
  });
  await report.step('Review order', async () => {
    await expect(page.getByTestId('summary')).toContainText('Express');
  });
});
```

Marker steps (no function)

Use a marker step to record a moment with a screenshot:

```
await page.goto('/pricing');
await report.step('Pricing page loaded');
```

Controlling the screenshot per step

```
// No screenshot for this step
await report.step('Wait for the API', () => page.waitForResponse('**/api/cart'), { screenshot: false });

// Screenshot only if this step fails
await report.step('Close the banner', () => page.getByLabel('Close').click(), { screenshot: 'on-failure' });

// Full scrollable page
await report.step('Scroll through terms', () =>
page.getByText('Terms').scrollIntoViewIfNeeded(), { fullPage: true });
```

Highlighting elements

highlight outlines the element in pink and dims the rest of the page in the screenshot. The highlight is removed right after.

```
// One element
await report.step('Price is shown', async () => {
  await expect(page.getByTestId('price')).toHaveText('$90');
}, { highlight: page.getByTestId('price') });

// Several elements
await report.step('Form has errors', async () => {
  await expect(page.getByText('Required')).toHaveCount(2);
}, { highlight: [page.getByLabel('Email'), page.getByLabel('Password')] });

// Every match of a locator
await report.step('All sale badges', () => Promise.resolve(), { highlight: page.locator('.badge-sale') });
```

Pointing errors at the caller (**box**)

When steps live in a helper function, **box: true** makes the error point at the line that called the helper, not the line inside it:

```
async function addToCart(page, report, name) {
  await report.step(`Add "${name}" to cart`, async () => {
    await page.getByRole('listitem').filter({ hasText: name }).getByRole('button', { name: 'Add' }).click();
  }, { box: true });
}
```

Steps in hooks

The `report` fixture used by `beforeEach`, the test and `afterEach` is the same object, so hook steps appear in the test's story:

```
test.beforeEach(async ({ page, report }) => {
  await report.step('Log in as admin', async () => {
    await page.goto('/login');
    await page.getByLabel('Email').fill('admin@example.com');
    await page.getByLabel('Password').fill(process.env.ADMIN_PASSWORD!);
    await page.getByRole('button', { name: 'Sign in' }).click();
  });
});

test.afterEach(async ({ report }) => {
  report.note('Test data cleaned up');
});
```

Mixing `report.step` and `test.step`

Both work in the same test. `report.step` calls appear in the **Test story** (with screenshots). `test.step` calls appear under **Technical steps**.

5. Screenshots

Automatic screenshots

When	Setting
After each <code>report.step()</code>	<code>capture.stepScreenshots: 'on'</code> (default)
When a test fails	<code>capture.failureScreenshot: true</code> (default)

Screenshot at any moment

```
await report.screenshot('Search results');
await report.screenshot('Whole page', { fullPage: true });
await report.screenshot('Checkout form', { highlight: page.getByTestId('checkout') });
```

Mobile and responsive checks

```
test('menu collapses on mobile', async ({ page, report }) => {
  await page.setViewportSize({ width: 390, height: 844 });
  await report.step('Open home page', () => page.goto('/'));
  await report.step('Hamburger menu is visible', async () => {
    await expect(page.getByRole('button', { name: 'Menu' })).toBeVisible();
  }, { highlight: page.getByRole('button', { name: 'Menu' }) });
});
```

Popups and new tabs

Screenshots use the test's main `page`. Switch pages with `report.usePage()`:

```
test('help opens in a new tab', async ({ page, report }) => {
  await page.goto('/');
  const popupPromise = page.waitForEvent('popup');
  await page.getByRole('link', { name: 'Help' }).click();
  const help = await popupPromise;

  report.usePage(help);
  await report.step('Help center opened'); // screenshot of the popup

  report.usePage(page);
  await report.step('Back on the main page'); // screenshot of the main page
});
```

You can also pass a page for a single step: `report.step(..., fn, { page: help })`.

Images from files

To show an existing image in the **Media** section, use Playwright's attach:

```
await test.info().attach('Generated invoice', { path: 'downloads/invoice.png', contentType: 'image/png' });
```

Visual comparisons

Playwright's `toHaveScreenshot` works as usual. When it fails, the report shows a **drag slider** comparing Expected and Actual, plus the Diff image:

```
await report.step('Home page matches the design', async () => {
  await expect(page).toHaveScreenshot('home.png', { maxDiffPixelRatio: 0.01, mask:
    [page.getByTestId('clock')] });
});
```

6. Videos and traces

Set these in the Aurora config:

```
capture: {
  video: 'retain-on-failure', // 'on' | 'off' | 'retain-on-failure' | 'on-first-retry'
  trace: 'retain-on-failure', // same values
}
```

In the report:

- **Video** plays inline with speed buttons (0.25×, 0.5×, 1×, 2×) and a download button.
- **Trace** shows a ready-to-copy `npx playwright show-trace "..."` command, a download button and a link to trace.playwright.dev.

Overriding per project (your `use` always wins over Aurora):

```
projects: [
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } }, // Aurora's
capture.video
  { name: 'mobile', use: { ...devices['Pixel 7'], video: 'on' } }, // always
record mobile
  { name: 'api', testMatch: /api\/.*\.spec\.ts/, use: { video: 'off', trace: 'off' } },
],
```

7. Soft checks, notes and data

Soft checks: keep going after a failure

`report.check()` records the result but doesn't stop the test. If any check failed, the test is marked failed at the end, with a list of every failed check.

```
test('product page content', async ({ page, report }) => {
  await page.goto('/product/42');

  await report.check('Title is correct', async () => {
    await expect(page.getByRole('heading')).toHaveText('Cloud Hoodie');
  });
  await report.check('Price is correct', async () => {
    await expect(page.getByTestId('price')).toHaveText('$60');
  });
  await report.check('Stock badge is shown', async () => {
    await expect(page.getByText('In stock')).toBeVisible();
  });
  // All three are checked even if the first one fails.
});
```

`check` returns `true` or `false`, so you can branch on it:

```
const hasPromo = await report.check('Promo banner visible', () =>
  expect(page.getByTestId('promo')).toBeVisible({ timeout: 2000 }));

if (hasPromo) {
  await report.step('Dismiss promo', () => page.getByTestId('promo-close').click());
}
```

By default a check only takes a screenshot when it fails. To always take one: `{ screenshot: 'on' }`.

Playwright's own `expect.soft()` also works. Each soft failure shows up as its own "Why it failed" card.

Notes

Notes are short messages in the story. There are three levels:

```
report.note('Using test account ada@example.com');
report.note('Feature flag NEW_CHECKOUT is enabled', 'success');
report.note('Payment sandbox is slow today; timeouts raised', 'warn');
```

Markdown-style ``code`` and `**bold**` are rendered:

```
report.note('Coupon `SAVE10` applied to **2 items**');
```

Attaching data

Data appears in the story as a collapsible preview:

```
// Object → pretty JSON
await report.attach('Order payload', { id: 991, items: ['hoodie', 'mug'], total: 85 });

// API response
const response = await request.get('/api/cart');
await report.attach('Cart API response', await response.json());

// Plain text
await report.attach('Generated SQL', 'SELECT * FROM orders WHERE id = 991');

// CSV with an explicit content type
await report.attach('Exported rows', 'id,name\n1,Hoodie\n2,Mug', 'text/csv');

// Test data for data-driven tests
await report.attach('Test data', user);
```

Text up to 64 KB is previewed inline. Larger or binary files are stored as files.

8. Test metadata: severity, owner, issues, tags

```
test('refund is issued within 5 days', async ({ page, report }) => {
  report.severity('critical');          // shows a colored badge: blocker/critical (red), high
                                        (amber), others neutral
  report.owner('Payments team');
  report.feature('Refunds');
  report.issue('PAY-1234');             // becomes a link when links.issue is configured
  report.link('https://wiki.example.com/refunds', 'Refund spec');
  report.description('Customers who cancel within 30 days get a full refund.\nThe refund must
show as Pending first.');
```

```
  // ...
});
```

Tags

Both Playwright ways of tagging work, and tags can be filtered in the report:

```
test('checkout works', { tag: ['@smoke', '@checkout'] }, async ({ page, report }) => { /* ... */
});

test('search works @smoke @search', async ({ page, report }) => { /* ... */ });

test.describe('Admin', { tag: '@admin' }, () => {
  test('can ban a user', async ({ page, report }) => { /* ... */ });
});
```

Metadata without the fixture

Annotations are read from any test, including plain `@playwright/test` ones:

```
test('legacy test', async ({ page }) => {
  test.info().annotations.push({ type: 'issue', description: 'JIRA-77' });
  test.info().annotations.push({ type: 'severity', description: 'high' });
  test.info().annotations.push({ type: 'owner', description: 'QA' });
});
```

Recognized annotation types: `severity`, `owner`, `feature`, `issue`, `bug`, `description`, `link`. Any other type (for example `slow`, `fixme` or your own) is shown as a plain badge.

Skipped tests show the reason

```
test('gift wrapping', async ({ page }) => {  
  test.skip(true, 'Ships in release 2.4');  
});  
  
test('Safari-only layout', async ({ page, browserName }) => {  
  test.skip(browserName !== 'webkit', 'Only relevant on Safari');  
});
```

9. Error capturing

Captured automatically (no code needed)

Every test that imports from `aurora-report` and uses `page` records:

What	Where in the report
Screenshot at the moment of failure	Story (last item: "Moment of failure") and Media
Page HTML at failure	Media → "Page HTML at failure"
Browser console errors and warnings	Browser diagnostics → Console
Uncaught JavaScript exceptions in the page	Browser diagnostics → Uncaught page errors
Failed requests and HTTP 4xx/5xx responses	Browser diagnostics → Failed network requests
Errors in popups and new tabs	Same tables

Diagnostics are recorded for **passing tests too**. A green test that throws page errors shows a red "⚠ 3 browser errors" badge, so hidden problems don't slip by.

Turn individual captures off in the config:

```
capture: { console: false, network: false, pageErrors: true }
```

Console messages: see everything that was logged

Every console error and warning is listed under **Browser diagnostics** → **Console**, one line each. Click a line to expand it:

Shown	Example
The full message	<code>Could not load recommendations {endpoint: ...} Error: ...</code>
Where it was logged	<code>http://localhost:3210/:88:14</code>
The page it happened on	<code>http://localhost:3210/</code>
Each value that was logged, separately	objects as pretty JSON, Errors with their stack, DOM elements as HTML

So an app that logs

```
console.error('Could not load recommendations', { endpoint, customer, cart }, err);
```

shows three values in the report: the text, the object as JSON, and the Error with its full stack trace — not `[object Object]`.

Uncaught page errors expand the same way, to the message and the stack.

Control what is recorded:

```
capture: {
  console: {
    levels: ['error', 'warning'], // add 'info', 'log', 'debug' to see more
    args: true, // read the logged values, not just the text
    maxArgSize: 4000, // each value is cut off after this many characters
    exclude: [
      'Download the React DevTools',
      '/third-party-widget/',
      (text) => text.startsWith('[HMR]'),
    ],
  },
},
```

`exclude` is matched against both the message and the URL it came from. Use `console: false` to turn console capture off entirely.

The browser logs its own `Failed to load resource` line for every failed request. If you exclude a URL from the **network** report, that echo is left out of the console list too, so one `exclude` entry keeps an API out of both.

Failed requests: what went out and what came back

Every failed request is recorded with both sides of the conversation, so you can debug an API failure without re-running anything:

Request	Response
Method and URL	Status and status text
Headers (secrets hidden)	Headers (secrets hidden)
Body / payload, JSON pretty-printed	Body, JSON pretty-printed
Resource type	How long it took

In the report they appear under **Browser diagnostics** → **Failed network requests**. Each row expands to show the request on the left and the response on the right.

"Failed" means an HTTP status of 400 or higher, or a request that never completed at all (DNS failure, connection refused, TLS error) — those show `FAILED` with the browser's error instead of a status.

Leaving noisy APIs out

Analytics beacons, pixels and third-party scripts fail all the time and tell you nothing about your app. Exclude them:

```
capture: {
  network: {
    exclude: [
      '**/analytics/**',          // * wildcard path
      'google-analytics.com',     // plain substring, matched anywhere in the URL
      'googletagmanager.com',
      /facebook\.com\/tr/,       // RegExp
      (url) => url.endsWith('.gif'), // your own rule
    ],
  },
},
```

Patterns can be:

Pattern	Matches
'checkout'	any URL containing "checkout"
'**/analytics/**'	any URL with an <code>/analytics/</code> path segment
'https://cdn.*.example.com/*'	one path segment under any <code>cdn.*</code> host
'//v1/(ads	beacon)//'
(url) => url.includes('?debug=')	anything you can express in code

To record **only** your own API instead, use `include`:

```
network: { include: ['api.my-app.com', '/api/'] }
```

`include` is applied first, then `exclude`. A request is recorded only if it passes both.

RegExp and function patterns must live in `aurora.config.js`, because they survive only when the config file is loaded by the worker. Options passed through `withAurora()` are serialised as JSON, so use string patterns there.

Keeping secrets out of the report

Authorization headers, cookies and API keys are replaced with `<hidden>` before anything is written. The default list is:

```
network: {
  redactHeaders: ['authorization', 'proxy-authorization', 'cookie', 'set-cookie',
    'x-api-key', 'x-auth-token', 'x-csrf-token'],
}
```

Add your own header names to extend it. Bodies are **not** redacted, so if your payloads contain secrets, turn the bodies off:

```
network: { requestBody: false, responseBody: false }
```

Bodies are cut off at `maxBodySize` (4 KB by default), and images, video, audio and font responses are never read.

How failures are explained

Aurora reads Playwright's error and turns it into a card with a plain-English title, an explanation, the Expected and Received values, the element, fix hints and the failing line of code.

Your code fails like this...	The report says...
<code>await expect(total).toHaveText('\$75')</code> and the page shows <code>\$85</code>	 A check did not match what the test expected: The test expected "\$75" but the app gave "\$85". (The difference is highlighted.)
<code>await page.getByRole('button', { name: 'Track order' }).click()</code> and the button doesn't exist	 An element never appeared on the page: The test waited 3s for <code>getByRole('button', { name: 'Track order' })</code> but it never showed up.
<code>await expect(toast).toBeVisible()</code> and the toast is hidden	 An element never appeared on the page: ...but it stayed hidden.
<code>page.getByRole('button').click()</code> matches 6 buttons	 The locator matched more than one element, with a hint to use a more precise locator
<code>await page.getByText('Save').click()</code> is blocked by a cookie banner	 The element was there but could not be used: Something else is covering it...
<code>await page.goto('http://localhost:3000')</code> and the server is down	 The page could not be reached (ERR_CONNECTION_REFUSED)
<code>await expect(page).toHaveScreenshot()</code> and pixels changed	 The page looks different from the approved screenshot, with the compare slider
The test exceeds its timeout	 The test ran out of time
<code>user.address.city</code> where <code>address</code> is undefined	 The test code itself threw an error
The page or browser closed mid-test (often a missing <code>await</code>)	 The browser or page closed unexpectedly

Click any error for full details

Errors are shown simply everywhere: one plain sentence, such as *"The test expected "\$75" but the app gave "\$85"."* **Click the error** to open the details panel. You can click it in any of these places:

- the **Why it failed** card (its header or the  **See full details** button)

- a failed step in the **Test story**
- a failed node in **Technical steps**
- the error line in the ► **Play story** player
- the error message of a group, or **Error details** on a test, on the **Failures** page

The panel explains the error in numbered sections:

Section	What it tells you
1. What was expected vs. what happened	Expected and Received side by side, with the difference highlighted
2. Where it happened	The failing step and when it ran, the element (with a Copy button), the wait time, the file and line, and the code around it
3. What the page looked like	The failure screenshot. Click it to page through all screenshots of the test.
4. What the test did just before	The last few steps and notes leading up to the failure
5. Step by step: what Playwright tried	Playwright's call log in plain words, such as "Looked for getByTestId('total')", "Read the value \"\$85\"..." and "repeated 9×"
6. Clues from the browser	Console errors, page errors and failed requests. Those logged during the failing step are highlighted.
7. Is this new?	Whether the test just started failing or keeps failing, a history strip, the retry results, and other tests that failed for the same reason
8. How to fix it	Concrete suggestions for this kind of failure
Technical details	The raw error message, call log and stack trace, collapsed by default

Buttons in the panel:

-  **Copy for bug report** copies a ready-to-paste Markdown summary (test, file, problem, failed step, expected and received, element, attempts and the raw error) for Jira, GitHub or Slack.
-  **Copy link** copies a link that opens the report straight to this error, for example `index.html#/tests/<id>?error=1.0`. The link format is `?error=<attempt>.<error>`, where `0` is the first run and `1` is the first retry.
- **Next error** > appears when a test has more than one error.

Close the panel with `Esc`, the **X** button, or a click outside it. With the keyboard, focus the **Why it failed** header and press `Enter`.

Write assertions that explain themselves

Custom messages make failures even clearer:

```
await expect(page.getByTestId('total'), 'Order total after coupon').toHaveText('$75');
expect(response.status(), 'Create-order API status').toBe(201);
```

Name steps after **what should be true**, not what the code does. The story then reads like a spec:

```
// ✗ Hard to read in a report
await report.step('click btn', ...);

// ✓ Easy to read
await report.step('Customer adds the hoodie to the cart', ...);
await report.step('Cart badge shows 1 item', ...);
```

Failures grouped by cause

Open the **Failures** tab to see failed tests grouped by root cause. If 12 tests fail because the login button was renamed, you see **one group with 12 tests** instead of 12 separate errors. Fix the biggest group first.

Groups are formed by the first line of the error, ignoring numbers, quoted text and URLs. The same kind of failure on *different* elements forms separate groups.

10. Real-world patterns

Page Object Model

Pass `report` into your page objects so every business action becomes a step:

```
// pages/LoginPage.ts
import type { Page } from '@playwright/test';
import type { Report } from 'aurora-report';
import { expect } from 'aurora-report';

export class LoginPage {
  constructor(private page: Page, private report: Report) {}

  async open() {
    await this.report.step('Open login page', () => this.page.goto('/login'));
  }

  async loginAs(email: string, password: string) {
    await this.report.step(`Log in as ${email}`, async () => {
      await this.page.getByLabel('Email').fill(email);
      await this.page.getByLabel('Password').fill(password);
      await this.page.getByRole('button', { name: 'Sign in' }).click();
    }, { box: true });
  }

  async expectError(message: string) {
    await this.report.step(`Error "${message}" is shown`, async () => {
      await expect(this.page.getByRole('alert')).toHaveText(message);
    }, { highlight: this.page.getByRole('alert') });
  }
}
```

```
// tests/login.spec.ts
import { test } from 'aurora-report';
import { LoginPage } from '../pages/LoginPage';

test('wrong password shows an error', async ({ page, report }) => {
  const login = new LoginPage(page, report);
  await login.open();
  await login.loginAs('ada@example.com', 'wrong');
  await login.expectError('Invalid email or password');
});
```

Custom fixtures on top of Aurora

```
// fixtures.ts
import { test as base, expect } from 'aurora-report';
import { LoginPage } from './pages/LoginPage';
import { CartPage } from './pages/CartPage';

type Pages = { loginPage: LoginPage; cartPage: CartPage };

export const test = base.extend<Pages>({
  loginPage: async ({ page, report }, use) => {
    await use(new LoginPage(page, report));
  },
  cartPage: async ({ page, report }, use) => {
    await use(new CartPage(page, report));
  },
});

export { expect };
```

```
// tests/cart.spec.ts
import { test, expect } from '../fixtures';

test('cart keeps items after reload', async ({ loginPage, cartPage, page, report }) => {
  await loginPage.open();
  await loginPage.loginAs('ada@example.com', 'secret');
  await cartPage.add('Cloud Hoodie');
  await report.step('Reload the page', () => page.reload());
  await cartPage.expectCount(1);
});
```

You already have your own fixtures file

Use `mergeTests`:

```
// fixtures.ts
import { mergeTests } from '@playwright/test';
import { test as auroraTest } from 'aurora-report';
import { test as dbTest } from './db-fixtures';
import { test as authTest } from './auth-fixtures';

export const test = mergeTests(auroraTest, dbTest, authTest);
export { expect } from 'aurora-report';
```

Reusable step helpers

```
// helpers/steps.ts
import type { Page } from '@playwright/test';
import type { Report } from 'aurora-report';
import { expect } from 'aurora-report';

export async function fillForm(report: Report, page: Page, fields: Record<string, string>) {
  await report.step('Fill in the form', async () => {
    for (const [label, value] of Object.entries(fields)) {
      await report.step(`${label}: ${value}`, () => page.getByLabel(label).fill(value), {
        screenshot: false });
    }
  });
}

export async function expectToast(report: Report, page: Page, text: string) {
  await report.step(`Toast "${text}" appears`, async () => {
    await expect(page.getByRole('status')).toHaveText(text);
  }, { highlight: page.getByRole('status'), box: true });
}
```

```
await fillForm(report, page, { 'First name': 'Ada', 'Last name': 'Lovelace', Email:
'ada@example.com' });
await expectToast(report, page, 'Profile saved');
```

Data-driven tests

```
const users = [
  { role: 'admin', email: 'admin@example.com', canSee: 'Settings' },
  { role: 'editor', email: 'editor@example.com', canSee: 'Drafts' },
  { role: 'viewer', email: 'viewer@example.com', canSee: 'Reports' },
];

for (const user of users) {
  test(`${user.role} sees the ${user.canSee} menu`, { tag: '@role-${user.role}' }, async ({
    page, report }) => {
    await report.attach('Test data', user);
    await report.step('Log in as ${user.role}', async () => {
      await page.goto('/login');
      await page.getByLabel('Email').fill(user.email);
      await page.getByLabel('Password').fill('secret');
      await page.getByRole('button', { name: 'Sign in' }).click();
    });
    await report.step(`${user.canSee} menu is visible`, async () => {
      await expect(page.getByRole('link', { name: user.canSee })).toBeVisible();
    }, { highlight: page.getByRole('navigation') });
  });
}
```

API tests (no browser)

`report` works without `page`. Steps are recorded without screenshots and no browser is launched:

```
test('create and fetch an order', { tag: '@api' }, async ({ request, report }) => {
  report.feature('Orders API');

  const created = await report.step('POST /api/orders', async () => {
    const res = await request.post('/api/orders', { data: { sku: 'HOODIE', qty: 1 } });
    expect(res.status()).toBe(201);
    return res.json();
  });
  await report.attach('Created order', created);

  await report.step('GET /api/orders/${created.id}', async () => {
    const res = await request.get(`/api/orders/${created.id}`);
    expect(await res.json()).toMatchObject({ sku: 'HOODIE', qty: 1 });
  });
});
```

API setup + UI check

```
test('order placed via API shows in the UI', async ({ request, page, report }) => {
  const order = await report.step('Create order through the API', async () => {
    const res = await request.post('/api/orders', { data: { sku: 'MUG', qty: 2 } });
    return res.json();
  }, { screenshot: false });

  await report.step('Open order history', () => page.goto('/orders'));
  await report.step(`Order #${order.id} is listed`, async () => {
    await expect(page.getByText(`#${order.id}`)).toBeVisible();
  }, { highlight: page.getByText(`#${order.id}`) });
});
```

Serial user journeys

```
test.describe.configure({ mode: 'serial' });

test.describe('New customer journey', () => {
  test('signs up', async ({ page, report }) => { /* ... */ });
  test('verifies email', async ({ page, report }) => { /* ... */ });
  test('places first order', async ({ page, report }) => { /* ... */ });
});
```

Authentication setup project

```
// tests/auth.setup.ts
import { test as setup, expect } from 'aurora-report';

setup('authenticate', async ({ page, report }) => {
  await report.step('Log in once for all tests', async () => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(process.env.E2E_USER!);
    await page.getByLabel('Password').fill(process.env.E2E_PASS!);
    await page.getByRole('button', { name: 'Sign in' }).click();
    await expect(page.getByText('Dashboard')).toBeVisible();
  });
  await page.context().storageState({ path: '.auth/user.json' });
});
```

```
// playwright.config.ts (projects)
projects: [
  { name: 'setup', testMatch: /\.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'], storageState: '.auth/user.json' },
dependencies: ['setup'] },
],
```

11. Configuration

You can set options in three places. When the same option is set in more than one place, the later one in this list wins:

1. `aurora.config.js` in your project root: good for team-wide settings.
2. `withAurora(config, { ... })`: good for values computed in the Playwright config.
3. **Reporter options** `['aurora-report/src/reporter.js', { ... }]`.

You can also pass JSON in the `AURORA_OPTIONS` environment variable for one-off runs (see [recipes](#)).

Run Playwright from the project root. Aurora looks for `aurora.config.*` in the current working directory.

Config file formats

Your project	Use
CommonJS (no <code>"type": "module"</code>)	<code>aurora.config.js</code> with <code>module.exports = { ... }</code>
ES modules (<code>"type": "module"</code>)	<code>aurora.config.cjs</code> with <code>module.exports = { ... }</code> , or <code>aurora.config.json</code>
Any	<code>aurora.config.json</code>

An ESM `export default` inside `aurora.config.js` **cannot be loaded**. Aurora prints a warning and uses the defaults, so use `.cjs` or `.json` in ESM projects.

Full example with every option

```
// aurora.config.js
const { defineAuroraConfig } = require('aurora-report'); // gives editor autocomplete

module.exports = defineAuroraConfig({
  title: 'Nimbus Store - E2E',
  subtitle: 'Nightly regression', // defaults to the run date
  logo: './assets/logo.svg', // file path (embedded) or https:// URL
  outputDir: 'aurora-report',
  live: false, // true = watch the report while the tests run
  timestampedRuns: false, // true = keep every run in its own <timestamp> folder
  keepRuns: 30, // how many run folders to keep (0 = keep them all)
  open: 'on-failure', // 'always' | 'never' | 'on-failure' (never opens when
  CI is set)
  singleFile: false, // true = one HTML file with every screenshot and video
  embedded

  theme: {
    mode: 'auto', // 'auto' (follows the OS) | 'light' | 'dark'
    accent: '#7c5cff', // your brand color
  },

  capture: {
    stepScreenshots: 'on', // 'on' | 'on-failure' | 'off'
    failureScreenshot: true,
    failureHtml: true,
    fullPage: false,
    video: 'retain-on-failure', // applied by withAurora()
    trace: 'retain-on-failure', // applied by withAurora()
    console: { // true = defaults, false = off
      levels: ['error', 'warning'], // add 'info', 'log', 'debug' for more
      args: true, // objects, Errors and stacks, not just text
      exclude: [], // messages to leave out
    },
    pageErrors: true,
    network: { // true = defaults, false = off
      failedStatus: 400, // responses at or above this count as failures
      requestHeaders: true, requestBody: true,
      responseHeaders: true, responseBody: true,
      maxBodySize: 4096, // bodies are cut off after this many bytes
      redactHeaders: ['authorization', 'cookie', 'x-api-key'],
      exclude: ['**/analytics/**', '/google-analytics/'], // noisy APIs to leave out
      include: [], // when set, only these are recorded
    },
  },
});
```

```

},

charts: {
  trend: true,                // results over time
  timeline: true,
  slowest: true,
  suites: true,               // results by file
  failureReasons: true,
  projects: true,             // results by project (only shown with 2+ projects)
},

history: {
  enabled: true,              // stored in <outputDir>/history.json
  keep: 30,                   // number of runs kept for trends and stability
},

slowTestThreshold: 10000,     // ms; slower tests get a 🐢 badge

environment: {                // shown on the Overview page
  'App version': '2.3.1',
  Environment: 'staging',
  URL: 'https://staging.example.com',
},

links: {
  issue: 'https://yourcompany.atlassian.net/browse/{id}', // report.issue('PAY-12') → link
  ci: 'https://ci.example.com/job/e2e/123',               // ↗ button in the header (auto-
filled on GitHub Actions)
},
});

```

JSON version

```

{
  "title": "Nimbus Store - E2E",
  "open": "never",
  "theme": { "accent": "#0ea5e9" },
  "capture": { "stepScreenshots": "on-failure", "video": "on-first-retry" },
  "links": { "issue": "https://github.com/acme/shop/issues/{id}" }
}

```

12. Configuration recipes

Different settings locally and on CI

```
// aurora.config.js
const isCI = !!process.env.CI;

module.exports = {
  title: 'Shop E2E',
  open: isCI ? 'never' : 'on-failure',
  capture: {
    stepScreenshots: isCI ? 'on-failure' : 'on', // faster CI runs, rich local debugging
    video: isCI ? 'retain-on-failure' : 'off',
    trace: isCI ? 'on-first-retry' : 'off',
  },
};
```

Show build info from CI variables

```
environment: {
  Branch: process.env.GITHUB_REF_NAME || process.env.BRANCH_NAME || 'local',
  Commit: (process.env.GITHUB_SHA || '').slice(0, 7) || 'local',
  'Triggered by': process.env.GITHUB_ACTOR || process.env.USERNAME || 'me',
  'Base URL': process.env.BASE_URL || 'http://localhost:3000',
},
```

One file to email or post in Slack

```
singleFile: true,
capture: { video: 'off', stepScreenshots: 'on-failure' }, // keeps the file small
```

Live mode: watch the report while the tests run

Normally the report appears when the run ends. In live mode Aurora serves it on localhost straight away and updates it as each test finishes — useful when a long suite is running and you want to look at the first failure without waiting.

```
live: true, // or: { port: 4321, open: true, hold: 0 }
```

Or without touching the config, for one run:

```
AURORA_LIVE=1 npx playwright test # bash
$env:AURORA_LIVE=1; npx playwright test # PowerShell
```

The run prints the address as it starts:

```
⚡ Aurora live report http://localhost:4321/
  updating as tests finish – the file report is written when the run ends
```

What you get:

- A **LIVE** badge in the header with a progress bar — `LIVE 8/20`.
- The headline reads *"Running... 8 of 20 tests done, 1 failed so far"*.
- Tests, stories, screenshots and failures appear as they complete. Screenshots are served from the run folder, so they work while the run is still going.
- Counts correct themselves: a test that fails and then passes on retry moves from failed to flaky in front of you.
- Open an error panel or the story player and updates pause rather than yanking the page around; they are applied when you close it.
- When the run ends the badge becomes **✓ Run finished** and the page holds the final results.

Option	Default	What it does
<code>port</code>	<code>4321</code>	First port to try; the next free one is used if it's taken
<code>open</code>	<code>true</code>	Open the live page in your browser when the run starts
<code>hold</code>	<code>0</code>	Seconds to keep the server up after the run ends, so you can keep browsing

```
live: { port: 5000, open: false, hold: 120 },
```

Notes:

- It listens on `127.0.0.1` only — nothing is exposed to your network.
- **Live mode never runs on CI** (it switches itself off when `CI` is set), and the file report is still written exactly as usual.
- With `hold: 0`, the command returns as soon as the run finishes; the page then shows the final data but stops updating. Use `hold` if you want to keep clicking around in the live page afterwards.

Keep every run instead of overwriting it

By default each run replaces the previous report. Turn on `timestampedRuns` to keep a history of reports on disk:

```
timestampedRuns: true,
keepRuns: 30,    // older folders are deleted; 0 keeps everything
```

Your report folder then looks like this:

```

aurora-report/
  index.html          ← list of every run, newest first
  history.json        ← shared, so trends still work
  2026-09-20_09-14-02/
    index.html        ← the report for that run
    results.json
    assets/           ← its own screenshots, videos and traces
  2026-09-20_11-47-35/
  2026-09-21_08-03-11/

```

- Folder names are local time, `YYYY-MM-DD_HH-MM-SS`, so they sort oldest to newest.
- Each folder is self-contained: copy one anywhere and it still works.
- `npx aurora-report open` opens the **newest** run; `npx aurora-report open --all` opens the list.
- Trends and stability keep working, because `history.json` stays at the top level and outlives pruned folders.

Handy on CI: publish the whole `aurora-report/` folder and every build keeps its own report.

Separate reports per environment

```

outputDir: `reports/aurora-${process.env.TEST_ENV || 'local'}`,
title: `Shop E2E - ${process.env.TEST_ENV || 'local'}`,

```

Fastest possible runs (smoke tests)

```

capture: {
  stepScreenshots: 'off',
  failureScreenshot: true,
  failureHtml: false,
  video: 'off',
  trace: 'off',
},
history: { enabled: false },

```

Company branding

```

title: 'Acme QA',
subtitle: 'Payments platform',
logo: './branding/acme-logo.svg',
theme: { mode: 'light', accent: '#e11d48' },

```

One-off overrides from the command line

PowerShell:

```
$env:AURORA_OPTIONS = '{"singleFile":true,"outputDir":"share-report"}'  
npx playwright test --grep "@smoke"  
Remove-Item Env:AURORA_OPTIONS
```

Command Prompt:

```
set AURORA_OPTIONS={"open":"always"}  
npx playwright test
```

bash:

```
AURORA_OPTIONS='{"theme":{"mode":"dark"}}' npx playwright test
```

Scripts in package.json

```
{  
  "scripts": {  
    "test:e2e": "playwright test",  
    "test:smoke": "playwright test --grep @smoke",  
    "test:debug": "playwright test --headed --workers=1",  
    "report": "aurora-report open"  
  }  
}
```

13. Running in CI

Aurora never opens a browser when the `CI` environment variable is set. Publish the `aurora-report` folder as a build artifact.

Trends and stability need `history.json` from earlier runs. CI machines start clean, so restore the file before tests and save it afterwards.

GitHub Actions

```
name: E2E
on: [push, pull_request]

jobs:
  e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps chromium

      - name: Restore report history
        uses: actions/cache/restore@v4
        with:
          path: aurora-report/history.json
          key: aurora-history-${{ github.ref_name }}-${{ github.run_id }}
          restore-keys: aurora-history-${{ github.ref_name }}-

      - run: npx playwright test
        env:
          BASE_URL: https://staging.example.com

      - name: Save report history
        if: always()
        uses: actions/cache/save@v4
        with:
          path: aurora-report/history.json
          key: aurora-history-${{ github.ref_name }}-${{ github.run_id }}

      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: aurora-report
          path: aurora-report/
          retention-days: 14
```

On GitHub Actions the header's CI link is filled in automatically.

Azure DevOps

```
steps:
  - script: npm ci && npx playwright install --with-deps chromium
  - script: npx playwright test
    env: { CI: 'true' }
  - task: PublishPipelineArtifact@1
    condition: always()
    inputs:
      targetPath: aurora-report
      artifact: aurora-report
```

Jenkins

```
stage('E2E') {
  steps {
    bat 'npm ci'
    bat 'npx playwright install chromium'
    bat 'npx playwright test'
  }
  post {
    always {
      archiveArtifacts artifacts: 'aurora-report/**', allowEmptyArchive: true
    }
  }
}
```

Sharding

Each shard writes its own report. **Aurora does not merge shard reports yet.** Give each shard its own folder:

```
// aurora.config.js
outputDir: process.env.SHARD ? `aurora-report/shard-${process.env.SHARD}` : 'aurora-report',
```

```
SHARD=1 npx playwright test --shard=1/3
SHARD=2 npx playwright test --shard=2/3
SHARD=3 npx playwright test --shard=3/3
```

14. Reading the report

Overview

- **Headline and summary:** the result in plain sentences. It names the most common failure cause, tests that started failing since the last run, tests that got fixed, and flaky tests.
- **Pass-rate ring:** click a segment to filter the Tests view.
- **Number tiles:** totals with ▲▼ changes since the previous run. Click a tile to filter.
- **What needs your attention:** failure groups and newly failing tests.
- **Charts:** results over time, results by file, slowest tests, why tests failed, and results by project. Hover for details and click to filter.

Tests

- A list with search, status filters, project and tag filters, sorting and a 🐢 slow-test filter.
- The small colored bars next to each test are its **last runs** (green passed, red failed, amber flaky).
- Any error can be clicked to open the **error details panel** (section 9).
- The detail panel, top to bottom: header and badges, **Why it failed**, **Test story** (with ► **Play story**), **Media and evidence**, **Browser diagnostics**, **Stability**, **Technical steps**, **Console output**.
- Tests that were retried have tabs: **First run**, **Retry 1**, and so on.

Failures

Failed tests grouped by cause, biggest group first, then flaky tests with the reason each one failed the first time.

Timeline

A Gantt chart with one row per worker. It also shows wall-clock time, total test time, the speed-up from parallel runs, and worker utilization. Low utilization usually means `workers` could be lowered, or your tests wait on shared resources.

Keyboard shortcuts

Key	Action
/ or Ctrl+K	Search
j / k or ↓ / ↑	Next or previous test
1 2 3 4	Overview, Tests, Failures, Timeline
← →	Previous or next screenshot in the story player
Space	Play or pause the story
Enter	Open error details (on a focused Why it failed header)
Esc	Close the error details, the player, or leave the search box

Links you can share

Add these after `index.html` to open a specific view:

Link	Opens
<code>#/overview</code>	Overview
<code>#/tests?status=failed</code>	Tests filtered to failures (<code>passed</code> , <code>flaky</code> , <code>skipped</code> also work)
<code>#/tests?tag=@smoke</code>	Tests with a tag
<code>#/tests?project=firefox</code>	Tests of one project
<code>#/tests?q=checkout</code>	Search results
<code>#/tests/<test-id></code>	One test (copy it from the address bar)
<code>#/tests/<test-id>?error=0.0</code>	One test with its error details panel open (use  Copy link in the panel)
<code>#/failures</code>	Failures grouped by cause
<code>#/timeline</code>	Timeline

15. Using results.json

Every run also writes `aurora-report/results.json` with all the data behind the report. Use it for chat notifications, dashboards or quality gates.

Print a summary

```
// scripts/summary.js
const { stats, clusters } = require('../aurora-report/results.json');

console.log(`Passed ${stats.passed}/${stats.total} (${Math.round(stats.passRate * 100)}%) in
${(stats.duration / 1000).toFixed(0)}s`);
for (const c of clusters) {
  console.log(`${c.icon} ${c.title} - ${c.tests.length} test(s): ${c.explanation}`);
}
```

Post to Slack or Teams

```
// scripts/notify.js (Node 18+)
const { stats, clusters } = require('../aurora-report/results.json');

const lines = [
  `*E2E ${stats.failed ? '❌ failed' : '✅ passed'}*: ${stats.passed} passed, ${stats.failed}
failed, ${stats.flaky} flaky`,
  ...clusters.slice(0, 3).map((c) => `${c.icon} ${c.title} (${c.tests.length})`),
  process.env.REPORT_URL ? `<${process.env.REPORT_URL}|Open report>` : '',
];

fetch(process.env.SLACK_WEBHOOK_URL, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ text: lines.filter(Boolean).join('\n') }),
});
```

Quality gate: fail the build if flakiness gets too high

```
// scripts/gate.js
const { stats } = require('../aurora-report/results.json');
const flakyRate = stats.flaky / Math.max(1, stats.total - stats.skipped);

if (flakyRate > 0.05) {
  console.error(`Flaky rate ${(flakyRate * 100).toFixed(1)}% is above 5%`);
  process.exit(1);
}
```

List failing tests with their explanation

```
const { tests } = require('../aurora-report/results.json');

tests
  .filter((t) => t.outcome === 'failed')
  .forEach((t) => {
    const err = t.attempts.at(-1).errors[0];
    console.log(`× [${t.project}] ${t.title} (${t.file}:${t.line})\n  ${err?.explanation}`);
  });
```

Useful fields:

Field	Contents
stats	total, passed, failed, flaky, skipped, duration, passRate, retries
tests[]	title, file, line, project, tags, outcome, severity, owner, feature, issues, duration, slow, attempts[]
tests[].attempts[]	status, duration, errors[] (title, explanation, hints, message, facts), story[], media, diagnostics
clusters[]	Failure groups: title, icon, explanation, message, tests[]
history[]	One entry per run: at, passed, failed, flaky, skipped, duration

16. Troubleshooting

No step screenshots

- Check that the spec imports `test` from `aurora-report`, not `@playwright/test`.
- The test must use the `page` fixture. API-only tests don't take screenshots, by design.
- Check `capture.stepScreenshots`. It may be `'off'` or `'on-failure'`.
- A parent step skips its own screenshot when a child step already took one.

No browser diagnostics or failure screenshot

- These need the `aurora-report` import. With plain `@playwright/test`, set `use: { screenshot: 'only-on-failure' }` to at least get Playwright's failure screenshot.

No video

- Check `capture.video`. Also check that `use.video` isn't set to `'off'` in your config or in a project, because your `use` wins.
- With `'retain-on-failure'`, passing tests have no video, by design.

[aurora] Could not load aurora.config.js warning

- Your project is ESM. Rename the file to `aurora.config.cjs` with `module.exports = {...}`, or use `aurora.config.json`.

Config file is ignored

- Run `npx playwright test` from the folder that contains `aurora.config.*`.

The report doesn't open automatically

- `open` defaults to `'on-failure'`. Set `open: 'always'` to open it every time.
- It never opens when the `CI` variable is set. Use `npx aurora-report open` instead.

"Results over time" shows only one bar

- Trends build up with each run. On CI, restore `history.json` between runs ([section 13](#)).
- Deleting the report folder resets history.

"Requiring @playwright/test second time"

- Aurora was installed with `npm link` or a folder link. Reinstall it from the `.tgz` ([section 1](#)).

The report folder is large

- Use `capture.video: 'retain-on-failure'` and `stepScreenshots: 'on-failure'`, and set `trace` to `'on-first-retry'`.
- With `timestampedRuns`, lower `keepRuns` so old run folders are deleted sooner.

Where are the files?

- `aurora-report/index.html` is the report — or, with `timestampedRuns`, the list of all runs.
- `aurora-report/<timestamp>/index.html` is a single run's report when `timestampedRuns` is on.
- `aurora-report/assets/` holds screenshots, videos and traces (inside the run folder when timestamped).
- `aurora-report/results.json` holds the data.
- `aurora-report/history.json` holds past runs, and always stays at the top level.

Add `aurora-report/` to `.gitignore`.

17. Cheat sheet

```
import { test, expect } from 'aurora-report';

test('name @tag', { tag: '@smoke' }, async ({ page, report }) => {
  // metadata
  report.severity('critical');          report.owner('Team');
  report.feature('Checkout');          report.issue('JIRA-1');
  report.description('What this test proves');
  report.link('https://docs.example.com', 'Spec');

  // steps
  await report.step('Do something', async () => { /* ... */ });
  await report.step('Marker with screenshot');
  await report.step('No screenshot', fn, { screenshot: false });
  await report.step('Highlight', fn, { highlight: page.getByTestId('x') });
  await report.step('Full page', fn, { fullPage: true });
  const value = await report.step('Returns a value', async () => 42);

  // soft checks
  const ok = await report.check('Keeps going on failure', async () => { /* expect(...) */ });

  // evidence
  await report.screenshot('Named screenshot', { highlight: page.locator('#cart') });
  report.note('Info'); report.note('Careful', 'warn'); report.note('Great', 'success');
  await report.attach('Payload', { any: 'json' });

  // other pages
  report.usePage(popup);
});
```

```
// playwright.config.js
module.exports = withAurora(defineConfig({ /* ... */ }, { title: 'My App' }));
```

```
npx playwright test          # run and generate the report
npx aurora-report open       # open the latest report
```